

Recording

Date	:	15/06/2026
Problem Specification	:	Find the nth Fibonacci number using recursion.
Assumption	:	Assume the user enters an integer value. Assume the value of n is small enough that the Fibonacci number can be stored in an integer.
Limitation	:	This program only accepts non-negative integers. Large values of n may cause stack overflow due to recursive calls. Fibonacci values may exceed the range of the int data type for large n.
Input	:	Number n
Processing	:	Calculate the nth Fibonacci number using a tail-recursive function.
Output	:	The Fibonacci number corresponding to n.
Algorithm	:	Main Method Step 1: Read the number n from the user. Step 2: Call fib(n). Step 3: Display the returned Fibonacci number. Step 4: End the program. Method: fib(n) Step 1: Call fib(n, 0, 1). Step 2: Return the result. Method: fib(n, a, b) Step 1: If $n < 0$, display an error and terminate. Step 2: If $n == 0$, return a. Step 3: Call fib(n-1, a+b, a). Step 4: Return the result of the recursive call.

Programme listing : Programme file **attached**

Test data and expected output :

1. Test data: $n=0$
Expected output: Fib(0):0
2. Test data: $n= 5$
Expected output: Fib(5):5
3. Test data: $n = 7$
Expected output: Fib(7):13
4. Test data: $n=-5$
Expected output: Invalid Number. Please enter a non-negative number.

Output obtained for test data :

1. Test data: $n=0$
Expected output: Fib(0):0
2. Test data: $n= 5$
Expected output: Fib(5):5
3. Test data: $n = 7$
Expected output: Fib(7):13
4. Test data: $n=-5$
Expected output: Exception in thread "main"
java.lang.IllegalArgumentException: Please enter a non-negative number.
at Fibonacci.fib(Fibonacci.java:5)
at Fibonacci.fib(Fibonacci.java:15)
at Fibonacci.main(Fibonacci.java:22)

Analysis : The number of operations required depends on the value of n .

Operation	Number of Times
+, -	n times
<	$n+1$ times
==	$n+1$ times
Recursive call	n times
Return	$n+1$ times
Loop	Not used

Since each recursive call reduces n by 1 until it reaches 0, the number of recursive calls grows linearly with n .

Running Time: $\Theta(n)$

Space Complexity: $\Theta(n)$

because n recursive stack frames are created.

Conclusion : This program successfully calculates the **n th Fibonacci number** using a recursive method. The user enters a non-negative integer, and the program returns the corresponding Fibonacci number. The implementation uses recursion with three variables (n , a , and b) and produces the correct result efficiently

Discussion : The running time of this algorithm is **$\Theta(n)$** because the recursive function is called once for each value from n down to 0. Therefore, the number of operations increases linearly as the input value increases.

The program also handles invalid input by displaying an error when a negative number is entered. Although the program works well for small and medium values of n , very large values may cause stack overflow because of the recursive calls. This limitation can be overcome by using an iterative approach or a larger data type such as long for storing larger Fibonacci numbers.